

CSC 220

Advanced Programming

FAHMIDA HAMID

WEEK 03, SEP 08



Will the code compile? Will it run?

```
public class ExceptionDemo {  
    Run | Debug  
    public static void main(String[] args) {  
  
        int[] numbers = {10, 20, 30};  
  
        System.out.println(numbers[5]);  
  
        System.out.println(x: "Program completed.");  
    }  
}
```

Problems in a Program

A program may have different kinds of problems.
They can be compile-time errors, runtime exceptions, or logical errors.

Problem



Something
is wrong
in the program.

Compile-time error

- Detected by the Java compiler before the program runs.
- The program will not compile.
- Usually caused by breaking the rules of the Java language (syntax or type errors).

Example:

```
int x = "hello";  
// Error: incompatible types
```

The compiler catches this mistake,
so the program does not run.

Runtime exception

- The program compiles successfully, but an error occurs while it is running.
- Caused by something unexpected happening during execution.
- Results in an exception (e.g. an error message and the program may stop).

Example:

```
int[] numbers = {1, 2, 3};  
numbers[10] = 2;  
// Exception: ArrayIndexOutOfBoundsException
```

The program compiles, but crashes
at runtime because index 10 does not exist.

Logical error

- The program compiles and runs, but produces an incorrect result.
- Caused by an error in the program's logic (the algorithm or formula).
- No error message is shown – the program runs, but the answer is wrong.

Example:

```
double area = 2 * Math.PI * r;  
// Should be Math.PI * r * r
```

The program runs, but uses the wrong formula,
so the result is not correct.

Different problems. Different stages. Different solutions.

Learn to recognize them!

```
public class ExceptionDemo {
```

Run | Debug

```
    public static void main(String[] args) {
```

```
        int[] numbers = {10, 20, 30};
```

```
        try {
```

```
            System.out.println(numbers[5]);
```

```
        } catch (Exception e) {
```

```
            //System.out.println(e.getMessage());
```

```
            System.out.println(x: "Something went wrong");
```

```
        }
```

```
        System.out.println(x: "Program completed.");
```

```
    }
```

```
}
```

Does the error still exist?

- Exception handling does not mean “prevent all errors”;
- It means **respond appropriately when exceptional situations occur.**

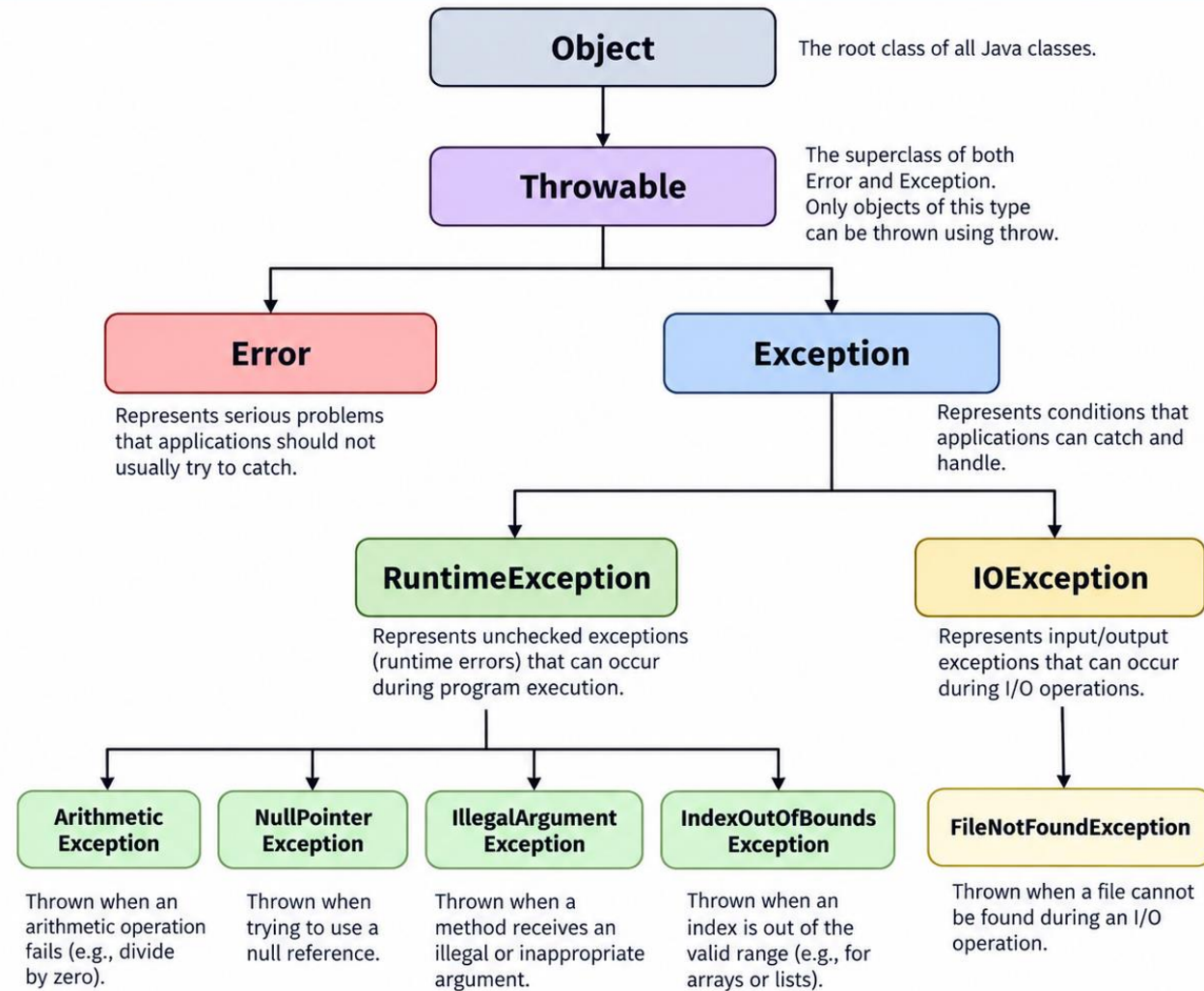
```
public static void main(String[] args) {  
    Scanner input = new Scanner(System.in);  
  
    try {  
        System.out.print(s: "Enter an integer: ");  
        int number = input.nextInt();  
        System.out.println(100 / number);  
    } catch (InputMismatchException e) {  
        System.out.println(x: "You must enter an integer.");  
    } catch (ArithmeticException e) {  
        System.out.println(x: "Cannot divide by zero.");  
    }  
    input.close(); // close a resource  
}
```

catch (Exception e)

- Why shouldn't we simply write catch (Exception e) for everything?

Java Exception Hierarchy

All Exceptions (and Errors) are Objects in Java



All Errors and Exceptions are Throwable, and all Throwables are Objects.

*Write Better Code
Handle Exceptions!*

catch (Exception e)

Because different exceptions indicate different problems.

Catching the **most appropriate/specific exception** allows our program to

- identify what went wrong,
- provide a meaningful message, and
- take the appropriate recovery action.

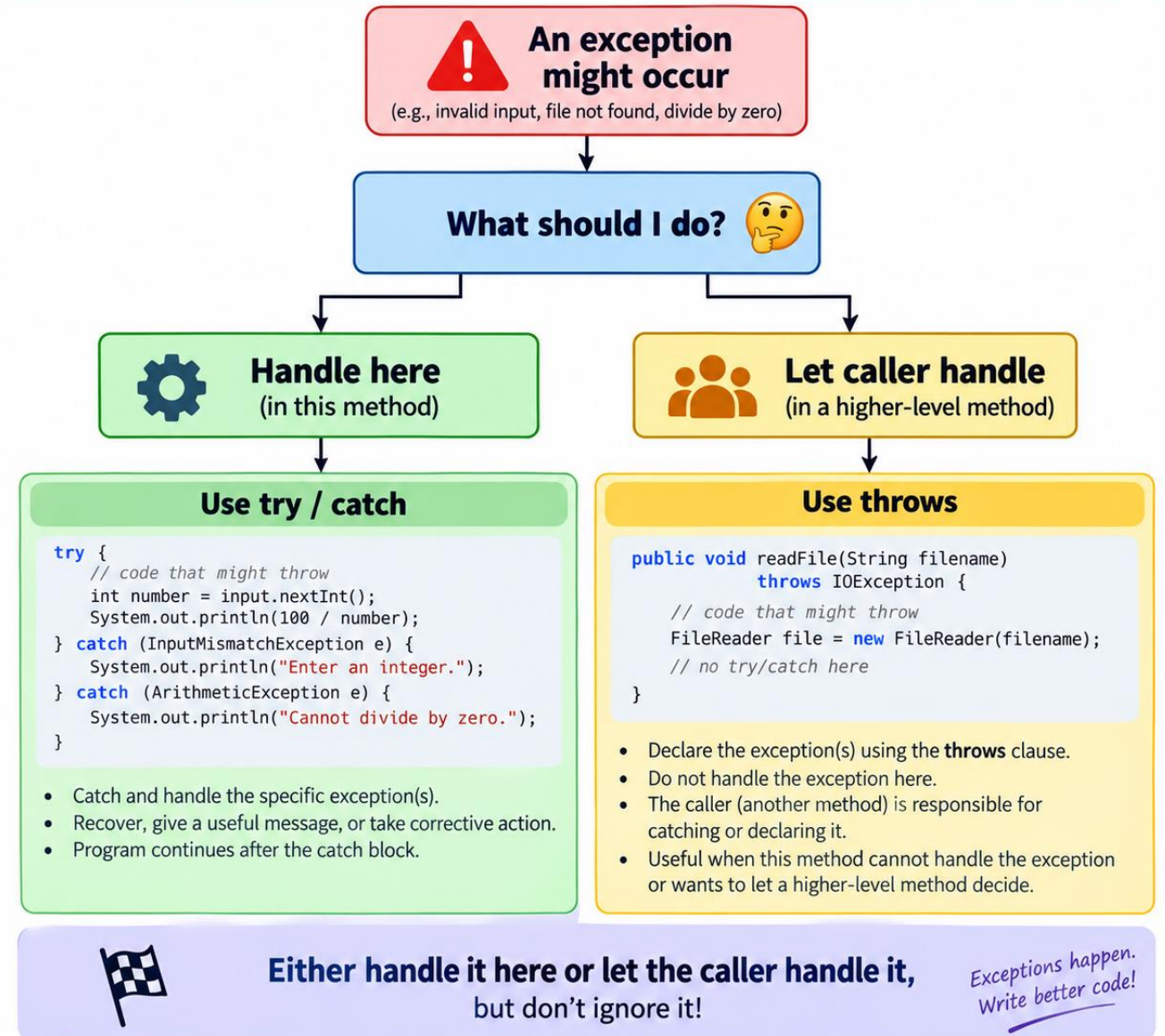
Ways to handle

...

Plan
Handle
Be Robust!

Handling Exceptions in Java

When an exception might occur, you have two choices!



Checked Exceptions

- Checked exception = compiler checks
- A **checked exception** is one that Java requires you to either:
 - **catch it**
 - declare it with **throws**
- If you do neither, the program **will not compile**.
- Example: IO Exceptions

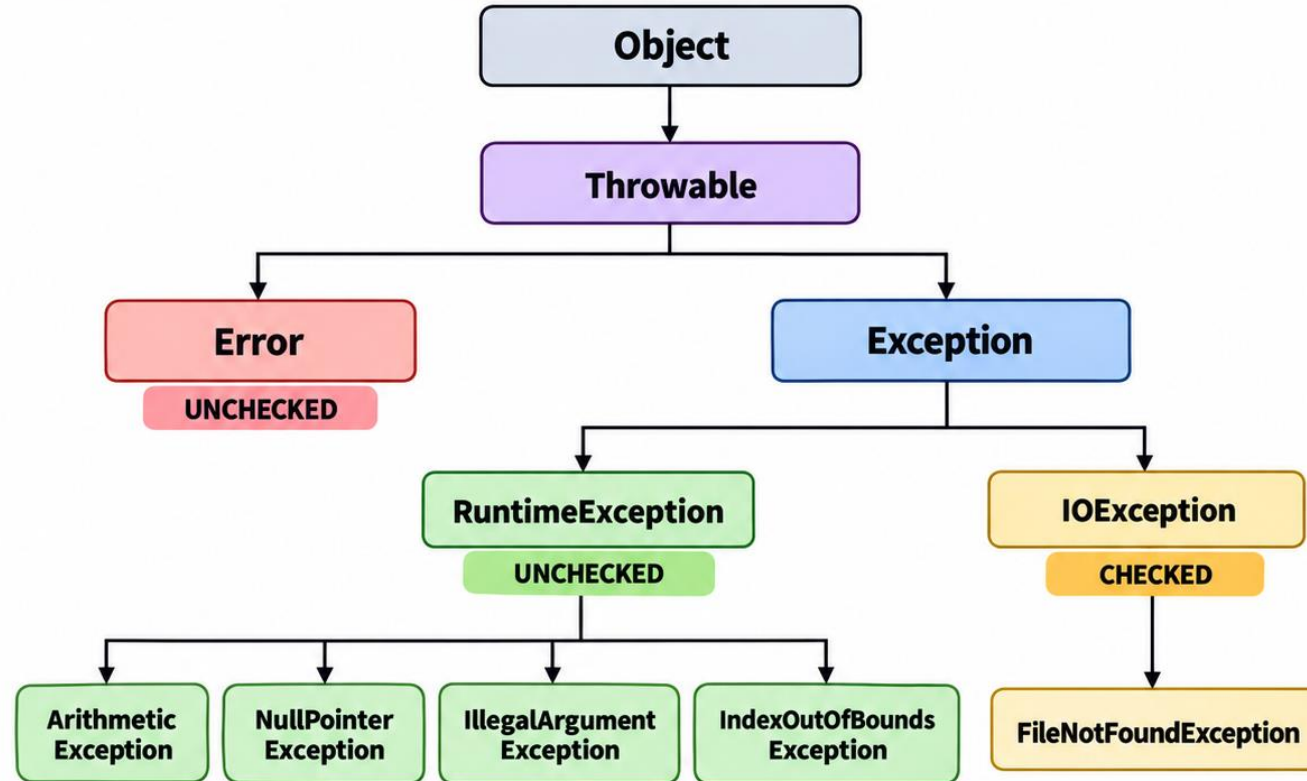
Unchecked exceptions

- ✓ Unchecked exception = compiler **does NOT force you**
- ✓ `String name = null;`
- ✓ `System.out.println(name.length());`

All exceptions
(and errors) are
objects in Java!

Java Exception Hierarchy

Checked vs. Unchecked Exceptions



Checked Exceptions

- Checked by the compiler (must be caught or declared with throws).
- Typically represent external or recoverable conditions (e.g., I/O operations).
- Examples: `IOException`, `FileNotFoundException`.

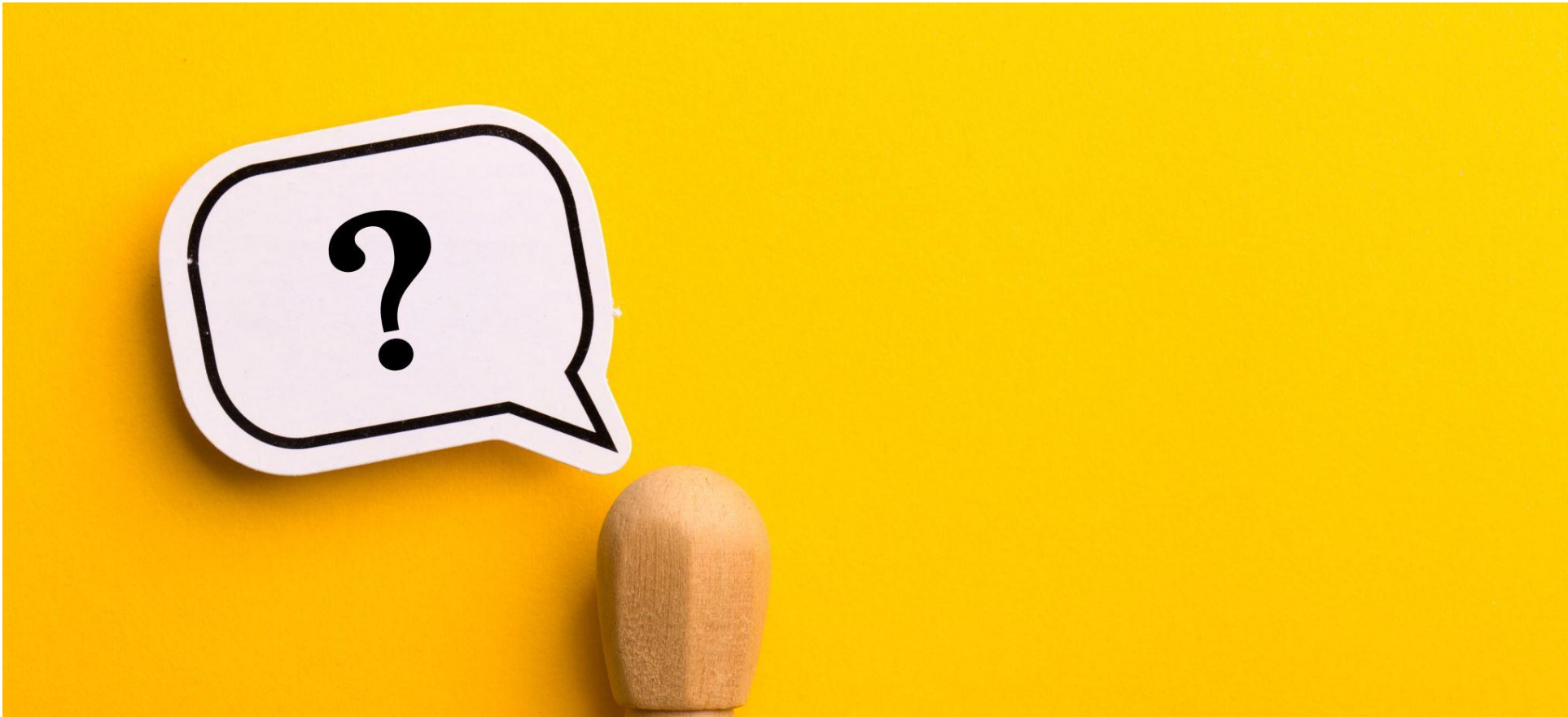
Unchecked Exceptions

- Not checked by the compiler (no requirement to catch or declare).
- Include `RuntimeException` and all its subclasses (and `Error`).
- Usually represent programming errors or invalid runtime conditions.
- Examples: `ArithmeticException`, `NullPointerException`, `IllegalArgumentException`, `IndexOutOfBoundsException`.

Use specific exception types to provide meaningful error handling!

*Better Code
More Reliable Programs!*

Possible cases where we might face exceptions/errors



In-class task

- **Exception Exploration Challenge**
 - **Working with your partner/group, use the official Java API documentation to investigate Java exceptions.**
- Create **3 small Java programs** that collectively demonstrate **at least 10 different exception types**.
- Across your three programs:
 - Demonstrate at least **10 distinct exceptions**.
 - Include both **checked and unchecked** exceptions.
 - For each exception, identify its name and explain **what caused it**.
 - Handle appropriate exceptions using try/catch.
 - Use **multiple catch blocks** in at least one program.
 - Use **finally** in at least one program.
 - Use **throws** in at least one method.
- For at least one exception, print information from the exception object
- using `getMessage()`, `getClass()`, or `printStackTrace()`.
- Include a comment identifying each exception as **checked or unchecked**.
- Be prepared to demonstrate one interesting exception to the class.

Sources you may use

- <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Exception.html>
- <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/RuntimeException.html>
- <https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html>